FORMAÇÃO FULL STACK · GUIA DE ESTUDO

JavaScript

do zero ao Sistema de Registro

Guia de revisão + resumo completo explicado em termos simples, com analogias, diagramas, perguntas para responder sem olhar e checklists, para construir o formulário, a API e a conexão com o MySQL da sua trilha.

A TRILHA EM 14 MÓDULOS

01 O que é JS
02 Variáveis
03 Condições
04 Laços
05 Funções
06 Arrays/Objetos
07 JSON
08 DOM
09 Validação
10 Assíncrono
11 Fetch/HTTP
12 Depuração
13 Node.js
14 Projeto 01

Material de estudo pessoal · baseado na trilha "Formação Full Stack — Sistema de Registro + MySQL"

Sumário

Primeiro Passo — Antes de começar . . . 3
Módulo 01 — O que é JavaScript . . . 5
Módulo 02 — Variáveis e tipos de dados . . . 7
Módulo 03 — Operadores e condições . . . 9
Módulo 04 — Laços de repetição . . . 11
Módulo 05 — Funções . . . 13
Módulo 06 — Arrays e objetos . . . 15
Módulo 07 — JSON . . . 18
Módulo 08 — DOM e eventos . . . 20
Módulo 09 — Validação de formulários . . . 23
Módulo 10 — JavaScript assíncrono . . . 25
Módulo 11 — Fetch API e HTTP . . . 27
Módulo 12 — Erros e depuração . . . 30
Módulo 13 — Ponte para o Node.js . . . 32
Módulo 14 — Projeto 01: tela de cadastro . . . 34
Referência — Plano de 14 dias e prova final . . . 37
Referência — Cola rápida (1 página) . . . 38
Referência — Glossário . . . 39
Referência — Gabarito das perguntas . . . 41

COMO USAR ESTE GUIA

Siga a ordem dos módulos: cada um constrói sobre o anterior. Os **Módulos 8, 9 e 11** são os que mais aparecem no seu projeto; o **Módulo 14** junta tudo. No final há o plano de 14 dias, a cola rápida, o glossário e o gabarito.

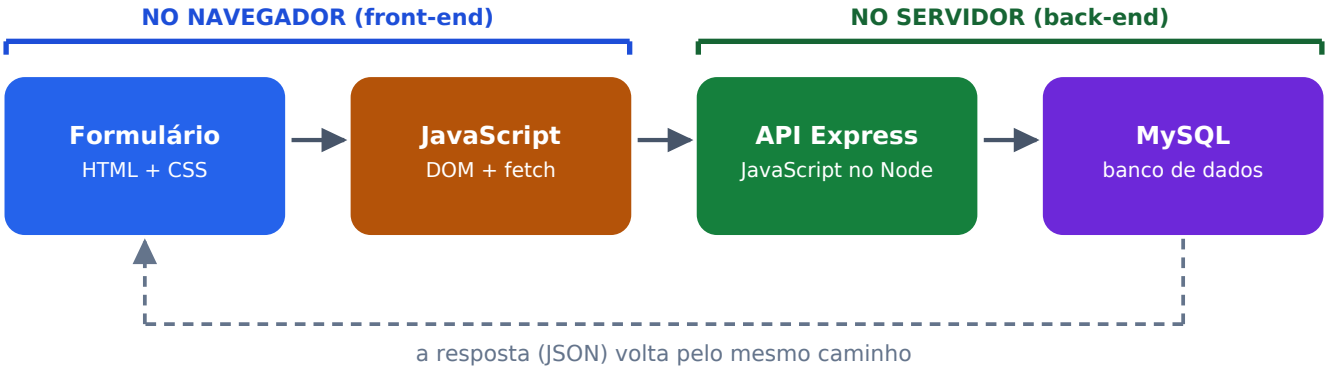
PRIMEIRO PASSO

Antes de começar

Mapa da jornada, ferramentas e como ler este guia

Para onde você vai

Seu objetivo é construir um **Sistema de Registro**: um formulário no navegador que envia dados para um servidor, que por sua vez grava tudo num banco MySQL. O detalhe importante: **JavaScript aparece nos dois lados**. No navegador ele faz o formulário "ganhar vida"; no servidor (Node.js) ele recebe os pedidos e conversa com o banco.



EM PORTUGUÊS CLARO

Este guia cobre o JavaScript necessário para os **Módulos 1 e 3** da trilha (JS inicial e JS aplicado) e termina com uma **ponte para o Node.js/Express** (Módulo 4). Cada módulo aqui tem a explicação em linguagem simples, código comentado, perguntas para responder sem olhar e um checklist de conclusão.

Como ler cada módulo

As caixas coloridas se repetem em todos os módulos. A cor diz o que a caixa é:

ANALOGIA	Uma comparação com o dia a dia para você "enxergar" o conceito antes de ver código.
EM PORTUGUÊS CLARO	O conceito resumido em poucas frases, sem jargão.
ATENÇÃO	Armadilhas e erros clássicos de quem está começando.
NO SEU PROJETO	Onde aquilo vai aparecer no Sistema de Registro.
RESPONDA SEM OLHAR	Perguntas de recuperação: responda de memória. Gabarito no final.
MINI-DESAFIO	Exercício para você escrever sozinho, sem copiar.
CHECKLIST DO MÓDULO	Só avance quando conseguir marcar todos os itens.

O método de cada módulo

1	**Teoria** Entenda o conceito e para que ele serve (leia a analogia e a explicação).
2	**Demonstração** Digite o código de exemplo no seu editor e rode. Mude valores e veja o que quebra.
3	**Exercício** Faça o mini-desafio sem abrir o exemplo.
4	**Revisão** Responda as perguntas sem olhar, confira no gabarito e anote as dúvidas.

> **ATENÇÃO**
> **Digite o código, não copie e cole.** Os dedos aprendem a sintaxe e você erra de verdade, que é onde o aprendizado acontece. Se um exemplo não rodar, leia a mensagem de erro com calma: o Módulo 12 ensina a decifrá-las.

Preparando o ambiente

1	**Instale o Visual Studio Code** É o editor onde você vai escrever tudo. Se já tem, não precisa reinstalar.
2	**Crie a pasta** `estudo-js` Dentro dela, dois arquivos: `index.html` e `script.js`.
3	**Abra o** `index.html` **no navegador** Dica: a extensão **Live Server** do VS Code recarrega a página sozinha toda vez que você salva.
4	**Abra o console do navegador** Aperte `F12` (ou clique com o botão direito → Inspecionar) e vá na aba **Console**.
5	**Node.js (só no Módulo 13)** Para conferir se já tem instalado: abra o terminal e digite `node -v`.

Use este `index.html` como base de todos os exercícios (o JS fica no arquivo `script.js`):

index.html

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Estudo de JavaScript</title>
</head>
<body>
  <h1>Estudo de JavaScript</h1>
  <p>Abra o console (F12) para ver os resultados.</p>

  <script src="script.js"></script>
</body>
</html>
```

MÓDULO 01

O que é JavaScript

A linguagem que faz a página (e o servidor) pensarem e agirem

ANALOGIA

Uma página web é como uma **casa**. O HTML é a estrutura (paredes, portas, janelas). O CSS é a decoração (cores, móveis, disposição). O **JavaScript é a parte elétrica e hidráulica**: faz a luz acender quando você aperta o interruptor, a torneira abrir, a campainha tocar. Sem ele a casa é bonita, mas nada responde a você.

EM PORTUGUÊS CLARO

JavaScript (JS) é uma linguagem de programação. Ele roda em **dois lugares**: dentro do navegador (para mexer na página) e no **Node.js** (para construir servidores). É a mesma linguagem nos dois, o que muda é o ambiente em volta.

Navegador × Node.js

	No navegador	No Node.js
Para que serve	Deixar a página interativa: validar formulário, reagir a cliques, pedir dados ao servidor	Construir o servidor: receber pedidos, regras de negócio, falar com o banco
Como roda	Automaticamente quando a página carrega	Você roda no terminal: `node arquivo.js`
Tem `document`/`window`?	Sim (a página inteira)	Não (não existe página)
Acessa arquivos e banco?	Não diretamente (por segurança)	Sim, com pacotes (ex.: `mysql2`)

Seu primeiro código

Todo programa começa com um jeito de **ver o que está acontecendo**. O `console.log()` escreve no console do navegador:

script.js

```
// script.js
console.log("Olá, mundo!");        // texto entre aspas
console.log(2 + 3);                // 5
console.log("Soma:", 2 + 3);       // vários valores separados por vírgula

/* comentário
   de várias linhas */
```

Regras básicas de escrita:

- JS diferencia maiúsculas de minúsculas: `nome` e `Nome` são variáveis diferentes.
- Terminar a linha com `;` é recomendado (o JS aceita sem, mas escolha um estilo e mantenha).
- Comentários começam com `//` (uma linha) ou ficam entre `/*` e `*/` (várias linhas).
- O `<script src="script.js">` vai **no final do** `<body>`, para o HTML já existir quando o JS rodar.

ATENÇÃO

JavaScript não tem relação com Java. Os nomes se parecem por marketing dos anos 90, mas são linguagens diferentes. Se você ouvir "Java" em um tutorial, não é isso que você está estudando.

NO SEU PROJETO

No Sistema de Registro, o JS do navegador vai validar o formulário e enviar os dados; o JS do Node vai receber esses dados e gravá-los no MySQL. Você vai escrever os dois, na mesma linguagem.

RESPONDA SEM OLHAR

1. Qual é a diferença entre JavaScript e Java?

2. Em quais dois lugares o JavaScript roda, e para que serve em cada um?

3. Por que o `<script>` fica no final do `<body>`?

4. Para que serve o `console.log()` e onde você vê o resultado?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

CHECKLIST DO MÓDULO

- ☐ Consigo explicar com minhas palavras o que o JS faz numa página.
- ☐ Sei abrir o console do navegador e imprimir uma mensagem.
- ☐ Tenho a pasta `estudo-js` com `index.html` e `script.js` funcionando.

MÓDULO 02

Variáveis e tipos de dados

Onde guardar informações e que formatos elas podem ter

ANALOGIA

Uma variável é uma **caixa de mudança com etiqueta**. A etiqueta é o nome; o que está dentro é o valor. Com `let`, você pode esvaziar a caixa e colocar outra coisa. Com `const`, a caixa é **lacrada**: depois que você coloca o valor, não troca por outro.

script.js

```
const nome = "Ana";            // não vai mudar
let tentativas = 0;             // vai mudar
tentativas = tentativas + 1;    // agora vale 1
tentativas++;                   // atalho: agora vale 2

console.log(nome, tentativas);  // Ana 2
```

EM PORTUGUÊS CLARO

Regra prática: **comece sempre com** `const`. Só troque para `let` quando perceber que o valor precisa mudar. Evite `var`: é a forma antiga e tem comportamentos confusos. Nomes seguem o padrão **camelCase**: `nomeUsuario`, `emailValido`.

Os tipos de dados

Tipo	O que guarda	Exemplo	`typeof` devolve
string	Texto	`"Ana"` `'oi'`	`"string"`
number	Números (inteiros e decimais)	`42` `3.14`	`"number"`
boolean	Verdadeiro ou falso	`true` `false`	`"boolean"`
null	"Vazio de propósito"	`null`	`"object"` (bug histórico)
undefined	"Ainda não definido"	`let x;`	`"undefined"`
array	Lista de valores	`[1, 2, 3]`	`"object"`
object	Ficha com campos nomeados	`{ nome: "Ana" }`	`"object"`

Texto: strings e template strings

script.js

```
const nome = "Ana";
const email = "  ANA@Email.com  ";

console.log(`Olá, ${nome}!`);               // Olá, Ana!   (crase + ${...})
console.log(email.trim());                    // "ANA@Email.com"  (tira espaços)
console.log(email.trim().toLowerCase());      // "ana@email.com"
console.log(nome.length);                     // 3
console.log(nome.includes("n"));              // true
console.log("formulario".slice(0, 4));        // "form"
```

Conversão de tipos

script.js

```
const digitado = "25";                // tudo que vem de um input é TEXTO

console.log(digitado + 5);            // "255"  (juntou texto!)
console.log(Number(digitado) + 5);    // 30
console.log(parseInt("12px"));        // 12
console.log(Number("abc"));           // NaN  (Not a Number: não é número)
console.log("5" - 3);                 // 2    (aqui o JS converteu sozinho)
```

ATENÇÃO

O JS tenta "adivinhar" conversões e isso gera bugs silenciosos. **Converta você mesmo**, de forma explícita, com `Number()`, `String()` ou `parseInt()`. E lembre: valor lido de um `input` é sempre string.

NO SEU PROJETO

Nome, e-mail e senha do formulário chegam como strings. Você vai usar `trim()` para limpar espaços, `toLowerCase()` para padronizar o e-mail e `length` para checar o tamanho da senha.

RESPONDA SEM OLHAR

1. Qual a diferença entre `const` e `let`? Qual usar por padrão?

2. Por que `"25" + 5` resulta em `"255"`?

3. O que é uma template string? Como colocar uma variável dentro dela?

4. Que tipo tem o valor lido de um `input` e como transformá-lo em número?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Crie `nome`, `email` (com espaços e maiúsculas) e `anoNascimento` (como string, simulando um input). Mostre no console: `Olá, <nome>! Seu e-mail limpo é <email>` e a idade aproximada no ano atual (converta o ano para número).

2. Teste de olho clínico: antes de rodar, preveja o resultado de `"10" + 2`, `"10" - 2` e `Number("")`. Depois rode e confira.

CHECKLIST DO MÓDULO

- ☐ Declaro variáveis com `const` e `let` sabendo a diferença.
- ☐ Reconheço os tipos string, number, boolean, null e undefined.
- ☐ Uso template strings com `${}`.
- ☐ Converto texto em número de forma explícita.
- ☐ Uso `trim()`, `toLowerCase()`, `includes()`, `length` e `slice()`.

MÓDULO 03

Operadores e condições

Como o programa toma decisões

ANALOGIA

Imagine um **porteiro de prédio**. Regra: entra quem tiver o crachá **E** o documento (&&). Ou então entra quem for morador (||). Se não cumprir nenhuma regra, ele manda esperar (`else`). Condições são exatamente esse tipo de regra de porteiro.

Os operadores principais

Operador	Significa	Exemplo → resultado
`+ - * /`	Soma, subtração, multiplicação, divisão	`10 / 4` → `2.5`
`%`	Resto da divisão	`10 % 3` → `1`
`===`	Igual (mesmo valor **e** mesmo tipo)	`5 === "5"` → `false`
`!==`	Diferente	`5 !== 6` → `true`
`> < >= <=`	Comparações	`10 >= 10` → `true`
`&&`	E (os dois precisam ser verdade)	`true && false` → `false`
`||`	OU (basta um ser verdade)	`true || false` → `true`
`!`	NÃO (inverte)	`!true` → `false`

if, else if, else

script.js

```
const email = "";
const senha = "123";

if (!email || !senha) {
  console.log("Preencha todos os campos");
} else if (senha.length < 8) {
  console.log("Senha muito curta");
} else {
  console.log("Tudo certo!");
}

// ternário: um if/else em uma linha
const forca = senha.length >= 8 ? "boa" : "fraca";
```

Valores "falsy" (que contam como falso)

EM PORTUGUÊS CLARO

Dentro de um `if`, estes valores se comportam como **falso**: `false`, `0`, `""` (texto vazio), `null`, `undefined` e `NaN`. Todo o resto é "verdadeiro". Por isso `if (!nome)` pega nome vazio, nulo ou indefinido.

ATENÇÃO

`=` **atribui,** `===` **compara.** `if (x = 5)` não compara nada: coloca 5 em `x` e sempre dá verdadeiro. Prefira `===` e `!==` e esqueça `==`, que converte tipos sozinho e dá resultados surpreendentes. E cuidado: `" "` (só espaços) é verdadeiro; use `trim()` antes de testar.

NO SEU PROJETO

Cada regra do formulário vira um `if`: nome vazio, e-mail inválido, senha curta. Na API, os mesmos `if` decidem se a resposta será de sucesso ou de erro.

RESPONDA SEM OLHAR

1. Qual a diferença entre `==` e `===`? Qual preferir?

2. Cite três valores "falsy".

3. O que `!nome || !email` significa em português?

4. Quanto valem `"10" === 10` e `10 % 3`?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Dada uma `nota`, mostre **Aprovado** (7 ou mais), **Recuperação** (de 5 a 6.9) ou **Reprovado**.

2. Dada uma `senha`, imprima uma mensagem para cada problema encontrado: menos de 8 caracteres e presença de espaços (`senha.includes(" ")`).

CHECKLIST DO MÓDULO

- ☐ Escrevo `if / else if / else` sem consultar.
- ☐ Combino condições com `&&`, `||` e `!`.
- ☐ Sei o que são valores falsy e uso `trim()` antes de testar texto.
- ☐ Uso `===` em vez de `==`.

MÓDULO 04

Laços de repetição

Como mandar o computador repetir tarefas sem copiar código

ANALOGIA

Pense em **voltas numa pista**. Você define: de onde começa (largada), até quando continua (enquanto não completar 5 voltas) e o que acontece a cada volta (soma 1). O `for` tem exatamente essas três partes.

script.js

```
for (let i = 1; i <= 3; i++) {      // início; condição; passo
  console.log("Volta", i);          // Volta 1, Volta 2, Volta 3
}

const nomes = ["Ana", "Bruno", "Carla"];
for (const nome of nomes) {         // para cada item da lista
  console.log(nome);
}

let contador = 3;
while (contador > 0) {              // enquanto a condição for verdadeira
  console.log(contador);
  contador--;                       // sem isto, o laço nunca termina!
}
```

Estrutura	Quando usar
`for (início; condição; passo)`	Quando você sabe quantas vezes repetir (ou precisa do índice `i`).
`for (const x of lista)`	Para percorrer **todos os itens** de uma lista, um por um.
`while (condição)`	Quando não sabe quantas voltas serão: repete até a condição virar falsa.
`break`	Sai do laço imediatamente.
`continue`	Pula para a próxima volta, ignorando o resto do corpo.

ATENÇÃO

Laço infinito acontece quando a condição nunca vira falsa (esquecer o `contador--`). A aba do navegador trava: feche-a. No terminal do Node, use `Ctrl + C`.

NO SEU PROJETO

Quando a API devolver 50 usuários, você vai percorrer a lista para criar uma linha de tabela por usuário. No Módulo 6 você verá `forEach` e `map`, versões mais elegantes, mas o raciocínio é o mesmo.

RESPONDA SEM OLHAR

1. Quais são as três partes do `for` clássico?

2. Quando usar `for...of` em vez do `for` clássico?

3. O que provoca um laço infinito?

4. Qual a diferença entre `break` e `continue`?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Imprima a tabuada do 7 (de 1 a 10) no formato `7 x 3 = 21`.

2. Some todos os números do array `[10, 20, 30, 40]` usando um laço e uma variável `total`.

3. Dado um array de nomes, conte quantos têm mais de 4 letras.

CHECKLIST DO MÓDULO

☐ Escrevo `for`, `for...of` e `while` de memória.

☐ Sei evitar laços infinitos.

☐ Consigo somar/contar itens de um array com um laço.

MÓDULO 05

Funções

Blocos de código reutilizáveis: a base de tudo que vem depois

ANALOGIA

Uma função é uma **receita de bolo**. Você dá os ingredientes (parâmetros), segue os passos e recebe o bolo pronto (`return`). Escrita uma vez, serve para quantos bolos você quiser, só trocando os ingredientes.

script.js

```
function somar(a, b) {          // a e b são PARÂMETROS
  return a + b;                  // devolve o resultado
}
console.log(somar(2, 3));        // 2 e 3 são ARGUMENTOS → 5

const dobro = (n) => n * 2;                        // arrow function
const saudar = (nome = "visitante") => `Olá, ${nome}!`;   // valor padrão

function emailValido(email) {
  return email.includes("@") && email.includes(".");
}
console.log(emailValido("ana@email.com"));  // true
console.log(saudar());                      // Olá, visitante!
```

Forma	Exemplo
Declaração	`function somar(a, b) { return a + b; }`
Função em variável	`const somar = function (a, b) { return a + b; };`
Arrow function (a mais usada hoje)	`const somar = (a, b) => a + b;`

EM PORTUGUÊS CLARO

Parâmetro é o nome que a função usa por dentro; **argumento** é o valor real que você passa na hora de chamar. `return` **devolve** um valor para quem chamou; `console.log` apenas **mostra** algo no console. São coisas diferentes!

Escopo: quem enxerga o quê

ANALOGIA

Uma casa com quartos. O que está na **sala** (fora de tudo) todos enxergam. O que está dentro de um **quarto** (dentro da função) só existe ali; de fora, ninguém vê. Mas de dentro do quarto você enxerga a sala.

script.js

```
const site = "Meu Sistema";        // "sala": global, todos enxergam

function mostrar() {
  const msg = "dentro da função";  // "quarto": só existe aqui
  console.log(site, msg);          // funciona
}

mostrar();
console.log(msg);                  // ERRO: msg is not defined
```

Callbacks: "me avise quando terminar"

Um **callback** é uma função que você entrega a outra função para ela chamar depois. É como deixar seu telefone na recepção: "ligue quando estiver pronto". Você vai ver isso o tempo todo em eventos, `setTimeout`, `map`, `filter` e rotas do Express.

script.js

```
setTimeout(() => {
  console.log("Passaram 2 segundos");   // roda DEPOIS
}, 2000);

console.log("Eu apareço primeiro!");    // roda ANTES
```

ATENÇÃO

Esquecer o `return` faz a função devolver `undefined`. E uma função com `{ }` na arrow precisa de `return` explícito; sem as chaves (`n => n * 2`) o retorno é automático.

NO SEU PROJETO

Você vai escrever funções como `validarCadastro(dados)`, `mostrarErros(erros)` e `cadastrarUsuario(dados)`. Funções pequenas, com uma responsabilidade cada, deixam o projeto fácil de testar e consertar.

RESPONDA SEM OLHAR

1. Qual a diferença entre parâmetro e argumento?

2. Qual a diferença entre `return` e `console.log`?

3. O que acontece se uma função não tiver `return`?

4. O que é um callback? Dê um exemplo.

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Escreva `calcularMedia(notas)` que recebe um array de números e devolve a média.

2. Escreva `validarSenha(senha)` que devolve `true` se tiver 8+ caracteres e `false` caso contrário. Depois a transforme em arrow function.

3. Escreva `ehMaiorDeIdade(idade)` e use-a dentro de um `if`.

CHECKLIST DO MÓDULO

- ☐ Escrevo função de declaração e arrow function.
- ☐ Sei a diferença entre parâmetro/argumento e entre `return`/`console.log`.
- ☐ Entendo escopo: o que existe dentro e fora de uma função.
- ☐ Sei explicar o que é um callback.

MÓDULO 06

Arrays e objetos

Como organizar listas e fichas de dados

ANALOGIA

Um **array** é uma prateleira com vagas numeradas a partir do **zero**. Um **objeto** é uma **ficha de cadastro**: cada campo tem um nome (nome, e-mail, ativo) e um valor. Uma lista de fichas (array de objetos) é praticamente uma **tabela de banco de dados**.

Tabela no MySQL

cada linha = 1 registro

id	nome	email
1	Ana	ana@e.com
2	Bruno	bru@e.com

⇄

Array de objetos no JS

cada objeto = 1 linha

```
[
  { id: 1, nome: "Ana",   email: "ana@e.com" },
  { id: 2, nome: "Bruno", email: "bru@e.com" },
]
```

Arrays

script.js

```
const frutas = ["maçã", "banana", "uva"];

console.log(frutas[0]);              // "maçã"  (primeira posição é 0!)
console.log(frutas.length);          // 3
frutas.push("pera");                 // adiciona no final
console.log(frutas.includes("uva")); // true
```

Objetos

script.js

```
const usuario = {
  id: 1,
  nome: "Ana",
  email: "ana@email.com",
  ativo: true,
};

console.log(usuario.nome);          // "Ana"        (notação de ponto)
console.log(usuario["email"]);      // "ana@email.com"  (colchetes)
usuario.ativo = false;              // altera um campo
usuario.cidade = "Rio";             // cria um campo novo

const { nome, email } = usuario;    // desestruturação: pega vários de uma vez
console.log(nome, email);
```

Array de objetos + os 4 métodos mais usados

script.js

```
const usuarios = [
  { id: 1, nome: "Ana",   ativo: true  },
  { id: 2, nome: "Bruno", ativo: false },
  { id: 3, nome: "Carla", ativo: true  },
];

usuarios.forEach((u) => console.log(u.nome));    // faz algo com cada item
const nomes  = usuarios.map((u) => u.nome);       // ["Ana", "Bruno", "Carla"]
const ativos = usuarios.filter((u) => u.ativo);   // só Ana e Carla
const bruno  = usuarios.find((u) => u.id === 2);  // o objeto do Bruno
```

Método	O que faz	Devolve
`forEach`	Executa uma ação para cada item	nada
`map`	Transforma cada item	novo array do mesmo tamanho
`filter`	Mantém só os itens que passam no teste	novo array (menor ou igual)
`find`	Acha o primeiro item que passa no teste	o item (ou `undefined`)
`push` / `pop`	Adiciona / remove no final	novo tamanho / item removido
`includes`	Verifica se o valor existe	`true` ou `false`

ATENÇÃO

Dois clássicos: **(1)** o primeiro item é o índice `0`, não `1`. **(2)** `const` impede *trocar* o array/objeto por outro, mas **permite alterar o conteúdo** (`push`, mudar um campo). Se um campo não existir, você recebe `undefined`, e acessar `.algo` de `undefined` dá `TypeError`.

NO SEU PROJETO

A tabela `usuarios` do MySQL chega ao navegador como **array de objetos**: cada linha é um objeto, cada coluna é uma propriedade. Listar, pesquisar e filtrar registros na tela será `map`, `filter` e `find` em cima desse array.

RESPONDA SEM OLHAR

1. Qual o índice do primeiro item de um array?

2. Qual a diferença entre `map` e `filter`?

3. De duas formas, como ler o campo `email` de um objeto `usuario`?

4. Por que `const lista = []` permite fazer `lista.push(1)`?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Crie um array de produtos `{ nome, preco }`. Calcule o total dos preços com um laço.

2. Com `filter` + `map`, gere só os **nomes** dos produtos com preço acima de 50.

3. Com `find`, ache um produto pelo nome e trate o caso de não encontrar (`undefined`).

CHECKLIST DO MÓDULO

- ☐ Crio arrays e objetos e leio/altero seus valores.
- ☐ Uso `forEach`, `map`, `filter` e `find` sem consultar.
- ☐ Entendo que linhas do banco viram objetos em um array.
- ☐ Sei usar desestruturação básica (`const { nome } = obj`).

MÓDULO 07

JSON

A "língua comum" entre navegador e servidor

ANALOGIA

Navegador e servidor são dois lugares distantes que **não compartilham memória**. Para trocar dados, precisam mandar uma "encomenda" em **embalagem padrão**, que os dois saibam abrir. JSON é essa embalagem: um texto com formato combinado. `JSON.stringify` **embala**; `JSON.parse` **desembala**.

script.js

```
const usuario = { nome: "Ana", email: "ana@email.com", ativo: true };

const texto = JSON.stringify(usuario);   // objeto → texto
console.log(texto);        // {"nome":"Ana","email":"ana@email.com","ativo":true}
console.log(typeof texto); // "string"

const volta = JSON.parse(texto);         // texto → objeto
console.log(volta.nome);   // "Ana"
```

Objeto JavaScript	JSON
Chaves podem ficar sem aspas: `{ nome: "Ana" }`	Chaves **sempre** entre aspas duplas: `{"nome":"Ana"}`
Aceita aspas simples	Só aspas duplas
Aceita funções, `undefined`, comentários	Só texto, números, booleanos, `null`, arrays e objetos
Vive na memória do programa	É **texto**: viaja pela rede e pode ser guardado em arquivo

ATENÇÃO

`JSON.parse` dá erro (`SyntaxError`) se o texto não for JSON válido, por exemplo quando o servidor devolve uma página HTML de erro. Em código real, envolva em `try/catch` (Módulo 10).

NO SEU PROJETO

No cadastro, você envia `JSON.stringify({ nome, email, senha })` e recebe a resposta com `await resposta.json()`, que já faz o `parse` por você.

RESPONDA SEM OLHAR

1. O que fazem `JSON.stringify` e `JSON.parse`?
2. Por que existe o JSON? Por que não enviar o objeto direto?
3. Cite duas diferenças entre um objeto JS e um JSON.
4. O que acontece se `JSON.parse` receber um texto inválido?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Crie um objeto com 3 campos de um usuário, converta em texto, imprima, converta de volta e altere um campo.
2. Rode `JSON.parse("{nome: 'Ana'}")`, leia o erro e corrija o texto para virar um JSON válido.

CHECKLIST DO MÓDULO

- ☐ Explico o que é JSON em uma frase.
- ☐ Converto objeto ↔ texto com `stringify` e `parse`.
- ☐ Sei apontar o que torna um JSON inválido.

MÓDULO 08

DOM e eventos

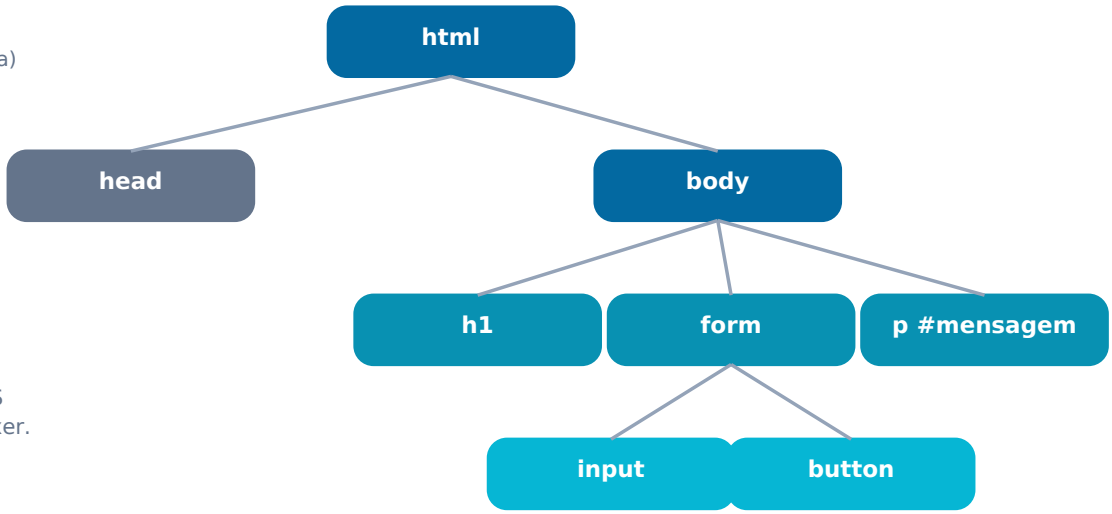
Como o JavaScript enxerga e mexe na página

ANALOGIA

O DOM é a **árvore genealógica da página**: o `html` é o ancestral, `head` e `body` são filhos, e assim por diante até cada botão e campo. O JS recebe um **controle remoto** dessa árvore: acha um elemento, lê o que tem nele, muda o texto, troca a cor e fica "escutando" para reagir quando o usuário clica.

document

(a árvore da página)



Cada caixa é um elemento que o JS pode achar e mexer.

EM PORTUGUÊS CLARO

DOM = a página representada como objetos que o JS consegue manipular. **Evento** = algo que acontece (clique, envio de formulário, tecla). **Handler** = a função que você pede para rodar quando o evento acontece.

As ações mais usadas

Quero...	Uso...
Achar um elemento	`document.querySelector("#id")` ou `(".classe")`
Achar vários	`document.querySelectorAll("li")`
Ler o que foi digitado	`input.value`
Mudar o texto	`elemento.textContent = "Olá"`
Mexer em classes CSS	`elemento.classList.add("erro")` / `.remove` / `.toggle`
Criar e inserir elemento	`document.createElement("li")` e `lista.append(item)`
Desabilitar um botão	`botao.disabled = true`
Reagir a um evento	`elemento.addEventListener("click", função)`

Exemplo 1: botão que conta cliques

index.html

```html
<button id="btn">Cliquei 0 vezes</button>
```

script.js

```js
const btn = document.querySelector("#btn");
let cliques = 0;

btn.addEventListener("click", () => {
  cliques++;
  btn.textContent = `Cliquei ${cliques} vezes`;
});
```

Exemplo 2: criar uma lista na página

script.js

```js
const lista = document.querySelector("#lista");   // <ul id="lista"></ul>
const nomes = ["Ana", "Bruno", "Carla"];

for (const nome of nomes) {
  const item = document.createElement("li");
  item.textContent = nome;    // seguro: trata como TEXTO, não como HTML
  lista.append(item);
}
```

Exemplo 3: formulário

index.html

```html
<form id="form-cadastro">
  <input type="text" id="nome" placeholder="Nome">
  <input type="email" id="email" placeholder="E-mail">
  <input type="password" id="senha" placeholder="Senha">
  <button type="submit">Cadastrar</button>
</form>
<p id="mensagem"></p>
```

script.js

```js
const form = document.querySelector("#form-cadastro");
const mensagem = document.querySelector("#mensagem");

form.addEventListener("submit", (evento) => {
  evento.preventDefault();      // impede a página de recarregar
  const nome = document.querySelector("#nome").value.trim();
  mensagem.textContent = `Olá, ${nome}!`;
});
```

ATENÇÃO

1. `Cannot read properties of null` quase sempre quer dizer que `querySelector` não achou o elemento (id errado ou `<script>` carregado antes do HTML).

2. Um `<form>` recarrega a página ao enviar; `evento.preventDefault()` evita isso.

3. Para mostrar dados de usuários use `textContent`. Com `innerHTML`, se alguém cadastrar um nome com código HTML/JS, ele será **executado** na página (falha chamada XSS).

NO SEU PROJETO

Este módulo é o coração do front-end: ler os campos do formulário, mostrar mensagens de erro/sucesso e montar a tabela de registros na tela.

RESPONDA SEM OLHAR

1. O que é o DOM?

2. Para que serve `evento.preventDefault()` no evento `submit`?

3. Por que usar `textContent` em vez de `innerHTML` para exibir dados?

4. Quais causas comuns do erro "Cannot read properties of null" com `querySelector`?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Faça um contador com três botões: **+**, **−** e **zerar**, mostrando o valor na tela.

2. Faça uma lista de tarefas: um `input` + botão que adiciona um `<li>`; clicar no item risca/desrisca (`classList.toggle`).

3. No formulário de cadastro, mostre a mensagem **"Cadastro recebido"** só se nenhum campo estiver vazio.

CHECKLIST DO MÓDULO

- ☐ Seleciono elementos por `id` e por classe.
- ☐ Leio `value` de inputs e altero `textContent`.
- ☐ Uso `addEventListener` com `click` e `submit`.
- ☐ Crio elementos com `createElement` e os coloco na página.
- ☐ Explico por que evitar `innerHTML` com dados de usuário.

MÓDULO 09

Validação de formulários

Conferir os dados antes de enviar (e por que isso não basta)

ANALOGIA

A validação no front-end é o **cartaz na recepção** ("preencha todos os campos"): ajuda a pessoa rapidamente. A validação no back-end é o **segurança na porta do cofre**: é quem realmente protege. Qualquer pessoa pode ignorar o cartaz (basta usar o Postman ou as ferramentas do navegador), mas não passa pelo segurança.

	Validar no front-end	Validar no back-end
Objetivo	Resposta rápida e boa experiência	Segurança e regras reais
Pode ser burlado?	Sim, facilmente	Não (o usuário não controla o servidor)
É obrigatório?	Recomendado	**Sempre**

Validação com atributos do HTML

index.html

```html
<input type="email" id="email" required>
<input type="password" id="senha" required minlength="8">
```

Ajudam, mas são básicos. Para mensagens personalizadas e para a lógica do projeto, você valida **com JavaScript**.

Validação com JavaScript

script.js

```javascript
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
console.log(emailRegex.test("ana@email.com"));   // true
console.log(emailRegex.test("ana@email"));       // false
console.log(emailRegex.test("ana email@x.com")); // false

function validar({ nome, email, senha, confirmar }) {
  const erros = [];
  if (!nome) erros.push("Nome é obrigatório");
  if (!emailRegex.test(email)) erros.push("E-mail inválido");
  if (senha.length < 8) erros.push("Senha precisa de 8+ caracteres");
  if (senha !== confirmar) erros.push("As senhas não conferem");
  return erros;     // lista vazia = tudo certo
}
```

EM PORTUGUÊS CLARO

A expressão `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` lê-se: *começo*, depois **qualquer coisa sem espaço nem @**, um `@`, de novo qualquer coisa sem espaço nem @, um **ponto**, mais um trecho e *fim*. Não é perfeita, mas pega os erros comuns.

ATENÇÃO

Limpe os dados antes de validar (`trim()`), nunca mostre a senha no console em código real, e lembre: **tudo que você validou no navegador precisa ser validado de novo no servidor** (Módulo 7 da trilha).

NO SEU PROJETO

O padrão "função `validar` devolve uma lista (ou objeto) de erros" é o que você vai usar no Projeto 01 e repetir no Express. A mesma ideia, nos dois lados.

RESPONDA SEM OLHAR

1. Por que também validar no back-end?

2. Que método de string remove espaços das pontas e por que usá-lo?

3. O que significa a função `validar` devolver uma lista vazia?

4. Cite dois atributos HTML de validação.

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Adicione o campo **confirmar senha** ao formulário e valide que é igual à senha.

2. Mostre cada erro ao lado do seu campo, em vermelho, usando uma classe CSS (`classList.add`).

3. Escreva `validarTelefone(texto)` que aceite só 10 ou 11 dígitos (dica: remova o que não for número antes de contar).

CHECKLIST DO MÓDULO

☐ Sei explicar a diferença entre validar no front e no back.

☐ Escrevo uma função que valida os campos e devolve os erros.

☐ Exibo mensagens de erro junto aos campos.

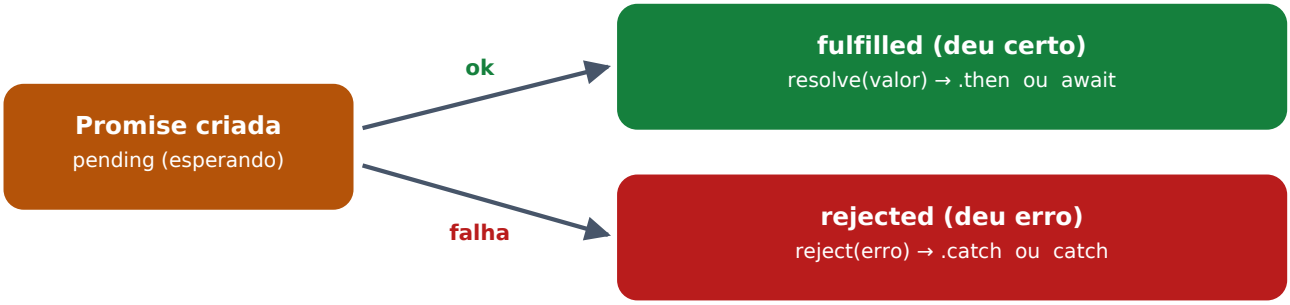
MÓDULO 10

JavaScript assíncrono

Promises, async/await e try/catch: esperar sem travar

ANALOGIA

Você pede um prato num **restaurante**. O garçom não fica parado na cozinha esperando: anota o pedido, te entrega uma **comanda** (a Promise) e vai atender outras mesas. Quando o prato fica pronto, ele avisa. O JavaScript tem **um garçom só**; se ele parasse para esperar cada pedido (pedir dados ao servidor, por exemplo), a página inteira congelaria.



EM PORTUGUÊS CLARO

Promise = uma promessa de resultado futuro. Pode estar *pending* (esperando), *fulfilled* (deu certo) ou *rejected* (deu erro). `async/await` é uma forma de escrever código assíncrono que **parece** código normal, de cima para baixo.

A ordem de execução surpreende

script.js

```
console.log("1 - início");
setTimeout(() => console.log("2 - timeout de 0 ms"), 0);
console.log("3 - fim");

// Saída: 1, 3, 2   (o que é assíncrono espera o código atual terminar)
```

async/await com tratamento de erro

script.js

```
const url = "https://jsonplaceholder.typicode.com/users";

async function carregar() {
  try {
    const resposta = await fetch(url);       // espera a resposta chegar
    const dados = await resposta.json();     // espera converter o JSON
    console.log(dados.length, "usuários");
  } catch (erro) {
    console.error("Deu ruim:", erro.message); // cai aqui se algo falhar
  } finally {
    console.log("Terminou (deu certo ou não)");
  }
}
carregar();
```

O mesmo, no estilo mais antigo (`.then`), que você ainda verá em muitos tutoriais:

```
script.js
```

```
fetch(url)
  .then((resposta) => resposta.json())
  .then((dados) => console.log(dados))
  .catch((erro) => console.error(erro));
```

Regra	Detalhe
`await` só funciona dentro de função `async`	Ou em módulos modernos; na dúvida, crie uma função `async`.
Função `async` sempre devolve uma Promise	Quem chama precisa de `await` (ou `.then`) para pegar o valor.
Esqueceu o `await`?	Você recebe `Promise { <pending> }` em vez do dado.
Use `try/catch`	Erros de rede ou do servidor viram exceções que você precisa tratar.

NO SEU PROJETO

Toda ida ao servidor (cadastrar, logar, listar, editar, excluir) é assíncrona. E no Node, toda consulta ao MySQL também. Este é o conceito que mais separa "mexer na página" de "fazer um sistema".

RESPONDA SEM OLHAR

1. Por que o JavaScript não "fica esperando parado" uma resposta do servidor?

2. Quais os três estados de uma Promise?

3. O que o `await` faz, onde pode ser usado e por que combinar com `try/catch`?

4. Qual a ordem das mensagens no exemplo do `setTimeout` com 0 ms?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Crie `esperar(ms)` que devolve uma Promise com `setTimeout`; use `await esperar(1000)` entre duas mensagens.

2. Busque os usuários da JSONPlaceholder e imprima só `nome - email` de cada um.

3. Troque a URL por uma inválida e veja o `catch` funcionar.

CHECKLIST DO MÓDULO

- ☐ Explico com a analogia do restaurante por que existe código assíncrono.
- ☐ Escrevo uma função `async` com `await` e `try/catch`.
- ☐ Sei prever a ordem de saída de código com `setTimeout`.

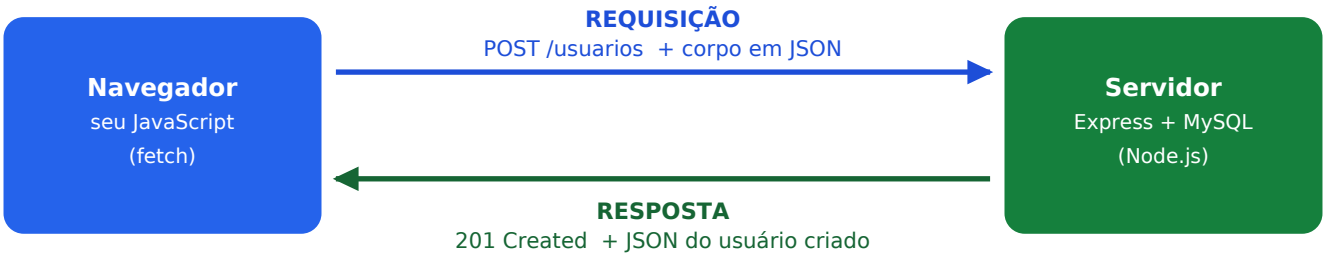
MÓDULO 11

Fetch API e HTTP

Como o navegador conversa com o servidor

ANALOGIA

Pense no **correio**. Você escreve uma carta com: o **endereço** (URL), o **tipo de pedido** (método: buscar, criar, apagar...), o **conteúdo** (corpo em JSON) e informações no envelope (headers). O servidor responde com outra carta: um **código** que diz como foi (status) e o **conteúdo** pedido. `fetch` é o carteiro.



Métodos HTTP ↔ CRUD ↔ SQL

Método HTTP	Ação (CRUD)	SQL equivalente	Exemplo de rota
`GET`	Ler (Read)	`SELECT`	`GET /usuarios`
`POST`	Criar (Create)	`INSERT`	`POST /usuarios`
`PUT` / `PATCH`	Atualizar (Update)	`UPDATE`	`PUT /usuarios/5`
`DELETE`	Excluir (Delete)	`DELETE`	`DELETE /usuarios/5`

Códigos de status que você vai ver

Código	Significado	Quando aparece
`200 OK`	Deu certo	Leitura/atualização ok
`201 Created`	Criado	Cadastro realizado
`400 Bad Request`	Pedido inválido	Dados que não passaram na validação
`401 Unauthorized`	Não autenticado	Login ausente ou inválido
`403 Forbidden`	Sem permissão	Logado, mas não pode fazer isso
`404 Not Found`	Não encontrado	Rota ou registro inexistente
`409 Conflict`	Conflito	E-mail já cadastrado (UNIQUE)
`500 Internal Server Error`	Erro no servidor	Falha no código/banco do back-end

Buscando dados (GET) e enviando dados (POST)

script.js

```
async function listarUsuarios() {
  try {
    const resposta = await fetch("http://localhost:3000/usuarios");
    if (!resposta.ok) throw new Error(`Erro ${resposta.status}`);
    const usuarios = await resposta.json();
    console.log(usuarios);
  } catch (erro) {
    console.error("Falha ao listar:", erro.message);
  }
}
```

script.js

```
async function cadastrarUsuario(dados) {
  const resposta = await fetch("http://localhost:3000/usuarios", {
    method: "POST",
    headers: { "Content-Type": "application/json" },   // "o corpo é JSON"
    body: JSON.stringify(dados),                       // objeto → texto JSON
  });
  const resultado = await resposta.json();
  if (!resposta.ok) throw new Error(resultado.mensagem || "Erro ao cadastrar");
  return resultado;
}
```

ATENÇÃO

`fetch` **só dá erro (`catch`) quando a rede falha.** Um `404` ou `500` chega como resposta normal; por isso você precisa checar `resposta.ok` (ou `resposta.status`). Esquecer isso é o erro número 1 de quem está começando.

ATENÇÃO: O ERRO MAIS COMUM NO PRIMEIRO PROJETO

CORS: por segurança, o navegador bloqueia seu front (ex.: `http://127.0.0.1:5500`) de ler respostas de um servidor de outra "origem" (ex.: `http://localhost:3000`), a menos que o servidor autorize. Se aparecer *blocked by CORS policy*, o problema **não está no seu fetch**: falta autorização no servidor. No Express resolve-se com o pacote `cors`.

Praticando sem back-end: API de teste

Enquanto o seu servidor não existe, você pode treinar `fetch` com uma API pública de testes, como a **JSONPlaceholder** (`jsonplaceholder.typicode.com`). Ela devolve dados de mentira e **simula** criações (não grava de verdade). É a "API de teste" citada na entrega do Módulo 3 da trilha.

script.js

```
async function buscarUsuariosDeTeste() {
  const resposta = await fetch("https://jsonplaceholder.typicode.com/users");
  const usuarios = await resposta.json();
  for (const u of usuarios) {
    console.log(u.name, "-", u.email);
  }
}
buscarUsuariosDeTeste();
```

NO SEU PROJETO

Este módulo é a **ponte** do projeto: o formulário (Módulo 8) + a validação (Módulo 9) + o `fetch` (este módulo) = tela de cadastro que conversa com a API. Entrega do Módulo 3 da trilha: formulário enviando dados para uma API de teste.

RESPONDA SEM OLHAR

1. Que método HTTP corresponde a cada operação do CRUD, e qual o SQL equivalente?

2. Por que checar `resposta.ok` depois do `fetch`?

3. O que significam os status 201, 400, 404 e 500?

4. Por que no POST precisamos de `Content-Type: application/json` e `JSON.stringify`?

5. O que é um erro de CORS e onde ele é corrigido?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Liste os usuários da JSONPlaceholder dentro de uma `<ul>` na página (Módulos 6 + 8 + 11 juntos).

2. Envie o formulário de cadastro por `POST` para `https://jsonplaceholder.typicode.com/posts` e mostre "Enviado! Status: 201" na tela.

3. Mude a URL para algo inexistente e mostre uma mensagem amigável ao usuário (sem usar `alert` com erro técnico).

CHECKLIST DO MÓDULO

- ☐ Faço um `GET` e um `POST` com `fetch` sem copiar.
- ☐ Verifico `resposta.ok` e trato erros com `try/catch`.
- ☐ Sei relacionar HTTP ↔ CRUD ↔ SQL.
- ☐ Sei reconhecer um erro de CORS e onde ele se resolve.

MÓDULO 12

Erros e depuração

Como ler mensagens de erro e achar o que deu errado

ANALOGIA

Depurar é ser **detetive**. A mensagem de erro é uma pista, não uma bronca: ela diz o **tipo** do problema, o que aconteceu e em qual **arquivo:linha**. Quem aprende a ler essas pistas resolve em minutos o que antes levava horas.

Mensagens que você vai encontrar

Mensagem	O que costuma ser	Onde olhar
`ReferenceError: x is not defined`	Variável que não existe (typo ou fora de escopo)	Nome da variável, escopo
`TypeError: Cannot read properties of null (reading 'value')`	`querySelector` não achou o elemento	Id/classe, posição do `<script>`
`TypeError: x is not a function`	Chamou como função algo que não é	Nome do método, parênteses
`SyntaxError: Unexpected token`	Faltou ou sobrou `)`, `}`, vírgula ou aspas	A linha apontada e a anterior
`Unexpected token '<' ... is not valid JSON`	Esperava JSON, recebeu HTML (URL errada, 404)	A URL e a aba Network
`Failed to fetch` / `blocked by CORS`	Servidor desligado, URL errada ou CORS	Servidor rodando? CORS liberado?
`Uncaught (in promise)`	Erro em código `async` sem `try/catch`	Adicionar `try/catch`

O ciclo do detetive

1. **Leia a mensagem inteira**
   Tipo do erro + descrição + arquivo e número da linha.
2. **Reproduza o erro**
   Descubra exatamente o que fazer para ele aparecer toda vez.
3. **Isole com `console.log`**
   Imprima valores antes da linha que quebra para ver o que realmente está lá.
4. **Mude uma coisa por vez**
   Se mudar cinco coisas juntas, não sabe qual resolveu.
5. **Anote a solução**
   Registre o erro e a correção na sua página do Notion (a trilha chama isso de "registrar dúvidas").

Ferramentas do navegador

script.js

```
const usuarios = [{ id: 1, nome: "Ana" }, { id: 2, nome: "Bruno" }];

console.log("usuarios:", usuarios);   // inspeciona qualquer valor
console.table(usuarios);              // mostra array de objetos como tabela
console.error("Aparece em vermelho"); // destaca problemas
debugger;                             // pausa aqui se o DevTools estiver aberto
```

- **Console:** mensagens, erros e o que você imprimiu.
- **Sources:** abre seu arquivo, permite colocar *breakpoints* (pausar numa linha) e andar passo a passo.
- **Network:** lista cada requisição `fetch`: URL, método, **status**, o que foi enviado (Payload) e o que voltou (Response). É a melhor ferramenta para o Módulo 11.

NO SEU PROJETO

Quando o cadastro "não funciona", 90% das vezes a resposta está na aba **Network** (o status e a resposta do servidor) ou no **Console** (erro de JS ou de CORS).

RESPONDA SEM OLHAR

1. Que três informações principais uma mensagem de erro traz?

2. O que "Cannot read properties of null" geralmente significa ao usar o DOM?

3. Onde você vê o status e o corpo da resposta de uma requisição?

4. Por que `Unexpected token '<'` aparece em `resposta.json()`?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Caça aos bugs: crie 5 códigos propositalmente errados (typo em variável, `}` faltando, id errado no `querySelector`, `await` fora de `async`, URL inválida), leia cada erro e conserte.

2. Faça um `fetch` para uma URL inexistente e descubra, só olhando a aba Network, qual foi o status.

CHECKLIST DO MÓDULO

- ☐ Leio a mensagem de erro até o fim antes de mexer no código.
- ☐ Uso `console.log` e `console.table` para investigar.
- ☐ Sei abrir a aba Network e ler status e resposta.

MÓDULO 13

Ponte para o Node.js

O mesmo JavaScript, agora no servidor

ANALOGIA

É a **mesma cozinha com outra clientela**. Você já sabe cozinhar (a linguagem). No navegador, atende quem está sentado no salão (a página). No Node, atende pedidos que chegam por telefone (requisições): não há salão, mas você ganha acesso à despensa (arquivos, rede, banco de dados).

Navegador	Node.js
Tem `document`, `window` e o DOM	Não tem DOM (não existe página)
Roda ao abrir a página	Roda no terminal: `node arquivo.js`
Código carregado por `<script>`	Usa módulos e pacotes do **npm**
Não acessa banco nem arquivos	Acessa arquivos, rede e banco (via pacotes como `mysql2`)

Um servidor mínimo (só para você ver que é JS)

Isto é prévia do Módulo 4 da trilha, onde você vai estudar com calma. No terminal, dentro de uma pasta nova: `npm init -y` e depois `npm install express`.

servidor.js

```
const express = require("express");
const app = express();

app.use(express.json());              // ensina o servidor a ler JSON do corpo

app.get("/", (req, res) => {          // rota GET /
  res.json({ mensagem: "API no ar" });
});

app.listen(3000, () => {
  console.log("Servidor em http://localhost:3000");
});
```

- `require("express")` traz o pacote que você instalou com o npm.
- `app.get("/", ...)` diz: quando alguém fizer **GET** em `/`, rode esta função (um **callback**!).
- `req` (request) é o pedido que chegou; `res` (response) é o que você devolve. `res.json()` já converte para JSON.
- `app.listen(3000, ...)` liga o servidor na porta 3000. Rode com `node servidor.js` e abra `http://localhost:3000`.

Tudo que você aprendeu reaparece aqui

Conceito de JS	Onde aparece no back-end
Funções e arrow functions	Handlers das rotas: `(req, res) => { ... }`
Objetos	`req.body` (dados recebidos) e `res.json({ ... })` (dados devolvidos)
JSON	`express.json()` lê o corpo; `res.json()` responde
`if` / validação	Responder `400` quando os dados são inválidos
`async/await` + `try/catch`	Consultar o MySQL e responder `500` se algo falhar
`map` / `filter`	Transformar as linhas devolvidas pelo banco

NO SEU PROJETO

Parabéns: com os Módulos 1 a 12 você já domina **a linguagem** necessária para os Módulos 4 a 6 da trilha (Node, Express, MySQL e autenticação). O que sobra a aprender é vocabulário novo (rotas, middlewares, `mysql2`), não lógica nova.

RESPONDA SEM OLHAR

1. Cite duas diferenças entre JavaScript no navegador e no Node.js.

2. O que é o npm?

3. Para que serve `app.use(express.json())`?

4. O que são `req` e `res` em uma rota do Express?

Escreva ou fale a resposta em voz alta. Só depois confira no gabarito (final do guia).

MINI-DESAFIO

1. Rode `node` num arquivo que imprime a soma de 1 a 10 (laço + `console.log`).

2. Suba o servidor mínimo, abra `http://localhost:3000` no navegador e veja o JSON.

3. Crie uma rota nova `GET /ola` que responda `{ mensagem: "Olá" }`.

CHECKLIST DO MÓDULO

- ☐ Rodo arquivos `.js` com `node` no terminal.
- ☐ Instalo um pacote com `npm install`.
- ☐ Subo um servidor Express mínimo e testo no navegador.
- ☐ Entendo onde cada conceito de JS aparece no servidor.

MÓDULO 14

Projeto 01: tela de cadastro

Juntando tudo: formulário, validação e confirmação

Esta é a primeira entrega da trilha: uma página com **nome, e-mail e senha** que valida os campos obrigatórios e mostra uma confirmação na tela. Ainda **sem banco e sem servidor**.

O fluxo que você vai programar

1	**Usuário clica em Cadastrar** Dispara o evento `submit` do formulário.
2	`preventDefault()` Impede a página de recarregar.
3	**Ler e limpar os valores** `value` de cada campo + `trim()`.
4	**Validar** Função que devolve os erros de cada campo.
5	**Se há erros** Mostra cada mensagem ao lado do campo e para.
6	**Se não há erros** Mostra a confirmação e limpa o formulário.

PRIMEIRO TENTE SOZINHO

Antes de olhar a solução, tente fazer sozinho. Primeiro monte o HTML (use o formulário do Módulo 8 e acrescente, abaixo de cada campo, um `<small id="erro-nome"></small>` e equivalentes). Depois o CSS. Só então o JS. Trave? Releia os Módulos 8 e 9. Quando terminar, **explique em voz alta** o que cada parte faz (último item do checklist da trilha).

Solução de referência (para comparar depois)

script.js

```
const form = document.querySelector("#form-cadastro");
const mensagem = document.querySelector("#mensagem");

function emailValido(email) {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function validarCadastro({ nome, email, senha }) {
  const erros = {};
  if (!nome) erros.nome = "Informe o nome.";
  if (!email) erros.email = "Informe o e-mail.";
  else if (!emailValido(email)) erros.email = "E-mail inválido.";
  if (senha.length < 8) erros.senha = "A senha precisa ter 8+ caracteres.";
  return erros;
}

function mostrarErros(erros) {
  for (const campo of ["nome", "email", "senha"]) {
    document.querySelector(`#erro-${campo}`).textContent = erros[campo] || "";
  }
}

form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const dados = {
    nome: document.querySelector("#nome").value.trim(),
    email: document.querySelector("#email").value.trim(),
    senha: document.querySelector("#senha").value,
  };
  const erros = validarCadastro(dados);
  mostrarErros(erros);
  if (Object.keys(erros).length > 0) {
    mensagem.textContent = "";
    return;                                  // para aqui se houver erro
  }
  mensagem.textContent = `Cadastro de ${dados.nome} recebido!`;
  form.reset();
});
```

Leitura guiada, bloco a bloco

Trecho	O que faz
`emailValido`	Testa o e-mail com a expressão regular do Módulo 9 e devolve `true`/`false`.
`validarCadastro`	Recebe os dados e devolve um **objeto de erros** (um campo por problema). Objeto vazio = tudo certo.
`mostrarErros`	Percorre os 3 campos e escreve a mensagem no `<small>` de cada um (ou limpa, com `|| ""`).
`evento.preventDefault()`	Evita o recarregamento do formulário.
`senha` sem `trim()`	Propositalmente: espaços numa senha são válidos e não devem ser removidos.
`Object.keys(erros).length`	Conta quantas chaves o objeto tem. Maior que 0 = há erros.
`return` dentro do handler	Interrompe a função: o código abaixo (a confirmação) só roda se não houver erro.
`form.reset()`	Limpa todos os campos do formulário.

Próximo degrau: enviar para uma API

Quando o formulário estiver funcionando, o passo seguinte (Projeto 02) é trocar a mensagem local por um `fetch` POST. O handler passa a ser `async`:

script.js

```
form.addEventListener("submit", async (evento) => {
  evento.preventDefault();
  // ... ler e validar como antes ...
  try {
    const resposta = await fetch("http://localhost:3000/usuarios", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(dados),
    });
    if (!resposta.ok) throw new Error(`Erro ${resposta.status}`);
    mensagem.textContent = "Cadastro realizado!";
  } catch (erro) {
    mensagem.textContent = "Não foi possível cadastrar. Tente de novo.";
  }
});
```

ATENÇÃO

Enviar a senha em texto puro como acima só é aceitável em **ambiente local de estudo**. Em produção usa-se HTTPS, e o **back-end** é quem transforma a senha em hash (com `bcrypt`) antes de gravar, como a trilha define. Nunca guarde senha pura no banco.

CHECKLIST DE CONCLUSÃO (DA TRILHA)

- ☐ Criei o HTML do formulário com nome, e-mail e senha.
- ☐ Estilizei com CSS.
- ☐ Capturo os valores com JavaScript.
- ☐ Valido os campos e mostro um erro ao lado de cada um.
- ☐ Mostro uma confirmação quando tudo está certo.
- ☐ Consigo explicar, linha por linha, como o código funciona.
- ☐ Refiz o projeto uma segunda vez **sem** olhar a solução.

REFERÊNCIA

Plano de 14 dias e prova final

Uma sugestão de ritmo e como saber se você está pronto

Este plano detalha o **Módulo 1 (JS inicial)** e o **Módulo 3 (JS aplicado)** da trilha. É uma referência, não uma obrigação: de 1 a 2 horas por dia basta. **Avance pela entrega, não pelo calendário**: se não conseguiu fazer o mini-desafio sem consultar, repita o dia.

Dia	Conteúdo	Você termina quando consegue...
1	Módulos 1 e 2	Rodar `console.log` e usar variáveis, tipos e template strings
2	Módulo 3	Validar uma senha com `if / else if / else`
3	Módulo 4	Somar e contar itens de um array com laços
4	Módulo 5	Escrever `calcularMedia` e `validarSenha` como funções
5	Módulo 6	Filtrar e transformar uma lista de objetos com `filter` e `map`
6	Módulo 7 + revisão dos dias 1 a 5	Responder as perguntas dos módulos 1 a 7 sem olhar
7	Módulo 8 (parte 1)	Contador de cliques e lista criada pelo JS
8	Módulo 8 (parte 2)	Formulário que lê campos e mostra mensagem
9	Módulo 9	Função `validar` com erros ao lado de cada campo
10	Módulo 14 (Projeto 01)	Tela de cadastro completa, **sem** consultar a solução
11	Módulo 10	`async/await` + `try/catch` com `setTimeout` e com `fetch`
12	Módulo 11	`GET` e `POST` na API de teste, checando `resposta.ok`
13	Módulos 12 e 13	Ler erros, usar a aba Network e subir um Express mínimo
14	Prova final	Fazer a prova abaixo, sem consultar

PROVA FINAL

Prova final (sem consulta): crie do zero uma página com formulário de cadastro (nome, e-mail, senha) que (1) valida os campos e mostra erros ao lado de cada um; (2) envia os dados por `POST` com `fetch` para a JSONPlaceholder; (3) mostra "Enviado! Status: 201" na tela; (4) trata falhas de rede com mensagem amigável; e (5) lista na mesma página usuários vindos de um `GET` (`map` + `createElement`). Se conseguir, você está pronto para o Módulo 4 (Node e Express).

PRONTO PARA O MÓDULO 4 QUANDO...

- ☐ Respondi as perguntas de todos os módulos sem olhar o gabarito (acertei a maioria).
- ☐ Fiz o Projeto 01 duas vezes, a segunda sem consulta.
- ☐ Fiz a prova final sem consultar este guia.
- ☐ Consigo explicar com minhas palavras o caminho formulário → fetch → API → banco.

REFERÊNCIA

Cola rápida (1 página)

Para consultar enquanto programa

Preciso de...	Sintaxe
Variável	`const nome = "Ana"; let n = 0;`
Texto com variável	`` `Olá, ${nome}` ``
Texto → número	`Number("25")` `parseInt("12px")`
Limpar texto	`texto.trim().toLowerCase()`
Condição	`if (a === b) { } else if (...) { } else { }`
E / OU / NÃO	`&&` `||` `!`
Laço	`for (let i = 0; i < n; i++) { }` `for (const x of lista) { }`
Função	`function f(a) { return a; }` `const f = (a) => a;`
Array	`push` `includes` `forEach` `map` `filter` `find` `length`
Objeto	`obj.campo` `obj["campo"]` `const { campo } = obj;`
JSON	`JSON.stringify(obj)` `JSON.parse(texto)`
Achar elemento	`document.querySelector("#id")`
Ler / escrever	`input.value` `el.textContent = "..."`
Evento	`el.addEventListener("click", () => { });`
Impedir recarregar	`evento.preventDefault();`
Classe CSS	`el.classList.add("erro")` `.remove` `.toggle`
Criar elemento	`const li = document.createElement("li");` `lista.append(li);`
Esperar	`async function f() { const x = await algo(); }`
Tratar erro	`try { } catch (erro) { } finally { }`
GET	`const r = await fetch(url);` `const d = await r.json();`
POST	`fetch(url, { method: "POST", headers: {...}, body: JSON.stringify(d) })`
Checar resposta	`if (!r.ok) throw new Error(r.status);`
Depurar	`console.log` `console.table` `debugger` DevTools → Network

REFERÊNCIA

Glossário

Os termos técnicos em português simples

Termo	Significado
API	Um "cardápio" de pedidos que um programa aceita receber de outros programas (ex.: "me dê a lista de usuários").
Argumento	O valor real que você passa ao chamar uma função.
Array	Lista ordenada de valores, com posições numeradas a partir de 0.
async / await	Jeito de escrever código assíncrono que parece sequencial: `await` espera a Promise terminar.
Back-end	A parte do sistema que roda no servidor: regras, banco de dados, segurança.
Booleano	Tipo que só aceita `true` ou `false`.
Callback	Função entregue a outra função para ser chamada depois.
CORS	Regra de segurança do navegador sobre quais sites podem ler respostas de um servidor.
Console	Janela do navegador (ou terminal) onde o JS mostra mensagens e erros.
CRUD	Criar, Ler, Atualizar e Excluir (Create, Read, Update, Delete): as 4 operações básicas sobre dados.
DOM	A página HTML vista como uma árvore de objetos que o JS pode manipular.
Endpoint / rota	Um endereço específico da API, como `/usuarios`.
Escopo	A região do código em que uma variável existe e pode ser vista.
Evento	Algo que acontece na página: clique, envio de formulário, tecla pressionada.
Fetch	Função do navegador para fazer requisições HTTP.
Front-end	A parte do sistema que roda no navegador: o que o usuário vê e usa.
Função	Bloco de código com nome que pode ser reutilizado.
HTTP	Protocolo (conjunto de regras) usado na web para pedir e responder dados.
JSON	Formato de texto para trocar dados entre sistemas.
Middleware	Função do Express que processa o pedido antes de chegar à rota (ex.: `express.json()`).
Node.js	Ambiente que permite rodar JavaScript fora do navegador, como servidor.
npm	Gerenciador de pacotes do Node: instala bibliotecas prontas.
Objeto	Estrutura com campos nomeados, como uma ficha de cadastro.
Parâmetro	O nome que a função usa por dentro para receber um valor.
Promise	"Comanda" de um resultado que ainda vai chegar: pode dar certo ou dar erro.
Requisição / Resposta	O pedido do navegador ao servidor e o que o servidor devolve.
Status code	Número que resume como foi a resposta (200, 404, 500...).

Termo	Significado
String	Tipo de dado para texto.
Template string	Texto entre crases que aceita `${variável}` no meio.
Validação	Conferir se os dados estão corretos antes de aceitar.

REFERÊNCIA

Gabarito das perguntas

Só confira depois de responder sem olhar

Módulo 01 · O que é JavaScript

1. São linguagens diferentes, sem relação técnica. O nome parecido foi uma decisão de marketing.
2. No navegador (deixa a página interativa) e no Node.js (constrói o servidor/back-end).
3. Para que os elementos HTML já existam quando o JS tentar achá-los. Se carregar antes, ele não encontra nada.
4. Escreve valores no console para você inspecionar o que o programa está fazendo. No navegador: F12 → aba Console.

Módulo 02 · Variáveis e tipos de dados

1. `const` não permite reatribuir o valor; `let` permite. Use `const` por padrão e `let` só quando o valor precisar mudar.
2. Com uma string envolvida o `+` concatena (junta texto) em vez de somar. É preciso converter com `Number()`.
3. Texto entre crases que aceita `${variável}` dentro, ex.: `` `Olá, ${nome}!` ``.
4. É sempre string. Converte-se com `Number(valor)` ou `parseInt(valor)`.

Módulo 03 · Operadores e condições

1. `==` converte tipos antes de comparar; `===` exige mesmo valor e mesmo tipo. Prefira `===`.
2. Qualquer três entre: `false`, `0`, `""`, `null`, `undefined`, `NaN`.
3. "Nome não preenchido OU e-mail não preenchido" (basta um dos dois faltar).
4. `false` (tipos diferentes) e `1` (resto da divisão).

Módulo 04 · Laços de repetição

1. Início (`let i = 0`), condição (`i < 5`) e passo (`i++`).
2. Quando só precisa percorrer todos os itens de uma lista, sem se importar com o índice.
3. Uma condição que nunca fica falsa, por exemplo esquecer de alterar a variável de controle.
4. `break` encerra o laço; `continue` pula só a volta atual e segue para a próxima.

Módulo 05 · Funções

1. Parâmetro é o nome na definição da função; argumento é o valor passado na chamada.
2. `return` devolve um valor para quem chamou; `console.log` só imprime no console e não devolve nada útil.
3. Ela devolve `undefined`.
4. Função passada como argumento para ser chamada depois. Ex.: a função dentro de `setTimeout` ou de `addEventListener`.

Módulo 06 · Arrays e objetos

1. 0 (zero).
2. `map` transforma cada item e mantém o tamanho; `filter` seleciona só os itens que passam no teste e pode diminuir o array.
3. `usuario.email` e `usuario["email"]`.
4. `const` só impede reatribuir a variável; o conteúdo do array continua podendo mudar.

Módulo 07 · JSON

1. `stringify` converte objeto em texto JSON; `parse` converte texto JSON em objeto.
2. Navegador e servidor são programas separados, sem memória compartilhada; precisam trocar dados num formato de texto padronizado.
3. Em JSON as chaves têm aspas duplas, não aceita funções/`undefined`/comentários e é um texto (string).
4. Lança um `SyntaxError`.

Módulo 08 · DOM e eventos

1. A representação da página HTML como uma árvore de objetos que o JavaScript pode ler e modificar.
2. Impede o comportamento padrão (recarregar a página) para que o JS trate o envio.
3. `textContent` trata como texto puro; `innerHTML` interpreta HTML e pode executar código malicioso (XSS).
4. Seletor/id errado, ou o `<script>` rodou antes de o elemento existir na página.

Módulo 09 · Validação de formulários

1. Porque a validação do navegador pode ser burlada; só o servidor é confiável.
2. `trim()`. Evita aceitar campos preenchidos só com espaços.
3. Que nenhum erro foi encontrado: os dados estão válidos.
4. `required`, `minlength`, `type="email"`, `maxlength`, `pattern` (qualquer dois).

Módulo 10 · JavaScript assíncrono

1. Porque tem uma única linha de execução; esperar travaria tudo. Por isso usa Promises/callbacks e continua o resto do código.
2. Pending (pendente), fulfilled (cumprida) e rejected (rejeitada).
3. Espera a Promise terminar antes de seguir; usa-se dentro de função `async`; `try/catch` trata falhas.
4. 1, 3, 2: o código assíncrono só roda depois que o código atual terminar.

Módulo 11 · Fetch API e HTTP

1. POST/INSERT (criar), GET/SELECT (ler), PUT ou PATCH/UPDATE (atualizar), DELETE/DELETE (excluir).
2. Porque `fetch` só rejeita em falha de rede; respostas 404/500 chegam normalmente e precisam ser tratadas.
3. 201 criado; 400 pedido inválido; 404 não encontrado; 500 erro interno do servidor.
4. Para avisar o servidor que o corpo é JSON e converter o objeto em texto JSON antes de enviar.
5. Bloqueio do navegador quando o servidor não autoriza a origem do front. Corrige-se no servidor (Express, com o pacote `cors`).

Módulo 12 · Erros e depuração

1. O tipo do erro, a descrição do problema e o arquivo/linha onde ocorreu.
2. Que o elemento procurado não foi encontrado (o seletor retornou `null`).
3. Na aba Network do DevTools (Payload e Response).
4. Porque a resposta era HTML (ex.: página de erro 404), não JSON; geralmente a URL está errada.

Módulo 13 · Ponte para o Node.js

1. No navegador há DOM e o código roda ao carregar a página; no Node não há DOM, roda no terminal e acessa arquivos/rede/banco.
2. O gerenciador de pacotes do Node: instala bibliotecas (como Express e mysql2) em seu projeto.
3. Para o Express ler e converter o corpo JSON das requisições em `req.body`.
4. `req` é a requisição recebida (dados do pedido); `res` é a resposta que o servidor devolve.